



TB-TBP: a task-based adaptive routing algorithm for network-on-chip in heterogeneous CPU-GPU architectures

Juan Fang¹ · Zhichao Wei¹ · Yaqi Liu¹ · Yumin Hou¹

Accepted: 30 September 2023 / Published online: 23 October 2023
© The Author(s) 2023

Abstract

With the rapid development of heterogeneous network-on-chip (NoC), a vast amount of shared resources are integrated into NoC. Intense resource competition exists between CPUs and GPUs, leading to congestion and a decrease in overall network performance. Reasonable node placement can minimize network conflicts at the topology level. This paper first discusses the placement of shared last-level cache and memory controller, then selects a more rational placement method and optimizes the path. To solve the hot spots problem in center placement method, a task-based routing algorithm is designed to plan the path. Simulation results demonstrate that, compared to the traditional routing algorithm, the overall network latency is reduced by 9%, and the CPU performance is improved by 13.6%. Furthermore, a dynamic task-based routing algorithm is proposed. Compared to the static task routing algorithm, the overall network latency is reduced by 2.08%, and the CPU performance is improved by 4.09%.

Keywords Heterogeneous architectures · Network-on-chip (NoC) · Routing algorithm · Task-based

Zhichao Wei, Yaqi Liu and Yumin Hou have contributed equally to this work.

✉ Juan Fang
fangjuan@bjut.edu.cn

Zhichao Wei
weizc@emails.bjut.edu.cn

Yaqi Liu
liuyq617@emails.bjut.edu.cn

Yumin Hou
houyumin@bjut.edu.cn

¹ Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China

1 Introduction

With the increasing demand for computational power in everyday life, homogeneous multi-core CPUs are no longer sufficient to meet the requirements [1]. In response to this, heterogeneous multi-core chips have emerged that integrate both CPU and GPU cores to meet the computational demand. Examples of such chips include Intel's Sandy Bridge [2] and AMD's Fusion APU chips [3], which have occupied a significant portion of the market. These products reduce communication overhead by sharing the same last-level-cache (LLC), memory controller (MC), and other on-chip resources between the CPU and GPU, thereby improving system performance. However, resource contention is a major challenge posed by resource sharing. The inherent characteristics of CPUs and GPUs result in differences in their shared resource utilization [4]. CPUs reduce memory access latency by utilizing multi-level caches [5], while GPUs can better tolerate latency problems by running a large number of parallel threads [6, 7]. Due to the requirement for extensive network resources to implement high parallel processing, the high throughput of GPUs may exacerbate the issues caused by resource sharing.

Network-on-chip (NoC) can be regarded as programmable systems for inter-node communication [8–10], connecting processing units with shared resources such as the LLC and MC. Proper management of the NoC, which is one of the largest shared resources in heterogeneous multi-core systems, is crucial to improving system performance. Although heterogeneous multi-core processors that integrate CPU and GPU cores theoretically offer higher peak performance, many factors such as data transmission between cores, resource allocation, and GPU programming strategies continue to affect the overall system performance. Due to the different computing characteristics of the two cores, the traditional cyclic scheduling allocation mechanism of NoC is likely to lead to significant performance losses as GPU occupies a large amount of network resources. To achieve reasonable resource allocation, it is necessary to separate the different traffic flows in the NoC of CPU-GPU heterogeneous systems.

The objective of this paper is to alleviate the contention of on-chip resources between CPU and GPU in a heterogeneous system. We first discuss the impact of different topologies on the performance of heterogeneous NoC, followed by proposing an optimized routing algorithm. **The contributions of this paper are summarized as follows:**

- We propose an LLC/MC CENTER architecture, which considers the impact of different LLC/MC placement methods on the performance of heterogeneous network systems in a mesh architecture.
- Based on the LLC/MC CENTER model, we analyze the easily congested paths on the network, optimize them based on different tasks in a high-traffic heterogeneous NoC, and propose a Task-Based (TB) routing algorithm to enhance the network's performance.
- We propose a Task-Based-Partition (TBP) routing algorithm that allocates the routing algorithms of different tasks into separate virtual channels. By utiliz-

ing dynamic monitoring technology to detect the phase behavior information of applications in the network, we propose an improved TB-TBP adaptive routing algorithm. The TB-TBP routing algorithm combines the advantages of both TB and TBP algorithms to enhance the system performance.

The remainder of this paper is organized as follows. Section 2 is Related Work. Section 3 analyzes the issue of performance degradation caused by mutual interference between CPU and GPU in heterogeneous multi-core systems and compares different LLC/MC placement methods. Section 4 elaborates on the proposed task-based routing algorithm and TB-TBP routing algorithm on the heterogeneous network architecture of LLC/MC CENTER. In Sects. 5 and 6, we present the simulator parameters for the experiments, the benchmark sets, and evaluation results and subsequent analysis. Section 7 concludes the article.

2 Related work

2.1 NoC routing algorithm

Some researchers have focused on optimizing the performance of NoC through communication routing mechanisms. Lee et al. [11] proposed a multicast routing scheme that guarantees deadlock-free and improved throughput by using router marking rules, destination router partitioning, and traffic-adaptive branching, ultimately reducing the number of packet hops and dispersing channel traffic. Elham et al. [12] developed a fault-tolerant routing algorithm (FT-PDC) based on path separation and congestion reduction for 3D NoC, which considers factors such as finding the shortest path, link failure, path diversity, and congestion. Alaei et al. [13] proposed a multicast adaptive routing algorithm for Mesh networks based on fuzzy load control, which dynamically prevents live-lock and deadlock situations using a fuzzy control system, effectively reducing latency and congestion. Salamat et al. [14, 15] proposed a high-performance adaptive routing algorithm for 3D NoC and conducted experiments on different traffic scenarios. Charles et al. [16] proposed a lightweight trust-aware routing mechanism that bypasses malicious IP cores during data packet transmission, reducing the number of retransmissions caused by data tampering, and minimizing the risk of DoS attacks, thereby improving NoC performance. Tang et al. [17] proposed a new routing performance metric called network pressure, and implemented high-performance routing based on network pressure and the Divide Conquer method. Kao et al. [18] demonstrated the potential of using reinforcement learning (RL) to optimize the runtime performance of NoCs, and proposed three RL-based optimization routing algorithms.

2.2 Heterogeneous NoC

Many researchers have turned to heterogeneous network-on-chip (NoC) to address the issue of on-chip resource contention between the CPU and GPU. Virtual channel

technology has been proposed as a deadlock avoidance solution in on-chip networks, which also helps prevent head-of-line blocking. Lee et al. conducted structural exploration on ring network and proposed an optimized placement method of ring heterogeneous network [19]. They also proposed a static and dynamic adaptive allocation method of the virtual channel in mesh network, targeting the virtual channel contention problem in heterogeneous NoC [20]. Cui et al. [21] introduced an interference-free NoC architecture by partitioning different routing nodes and placing them appropriately, enabling routing in different dimensions for different tasks. They reduced network energy consumption through dedicated routing algorithms and bypass techniques. Li et al. [22] optimized the ALPHA router for heterogeneous multicore systems by increasing the injection link width, crossbar size, and modifying the buffer organization of router injection ports to enhance injected bandwidth and improve throughput. It effectively addressed local and global contention to reduce network latency. Yin et al. [23] leveraged the diversity of on-chip heterogeneous computing devices and partitioned the network using time-division multiplexing, allowing packet-switched and circuit-switched messages to share the same communication structure, with circuit-switched paths established along frequently communicating nodes. Zhan et al. [24] designed the OSCAR architecture, which maximized the potential of STT-RAM-based LLC. They proposed an integrated asynchronous batch scheduling and priority-based on-chip network interconnect method. Fang et al. [25] first evaluated the placement of different buffered and unbuffered routers and proposed a Unidirectional Flow Control method to avoid the network congestion problem.

Most of the aforementioned research focused on homogeneous architectures, with limited exploration on networks with multiple types of computing cores. However, different cores have unique memory access characteristics, making it difficult to design a routing algorithm for one type of core to adapt to other types. At the same time, for heterogeneous on-chip networks, it is necessary to consider different task properties and design network topology and routing algorithms reasonably.

3 Background and motivation

3.1 Topology

The scalability and reliability of the network-on-chip (NoC) make it a suitable structure for heterogeneous systems. However, urgent attention is required to address issues such as network delay and performance degradation caused by contention for resources among different cores. In the general mesh heterogeneous network topology, each router is connected to an IP node, which can be CPU, GPU, LLC, MC, or other accelerators. This paper adopts the mesh architecture shown in Fig. 1, where routers are connected to CPU, GPU, LLC, and MC, respectively. The CPU features dual private caches, while the GPU has a single private cache. As the load traffic on the heterogeneous NoC increases, conflicts will intensify, and the network's latency will increase correspondingly.

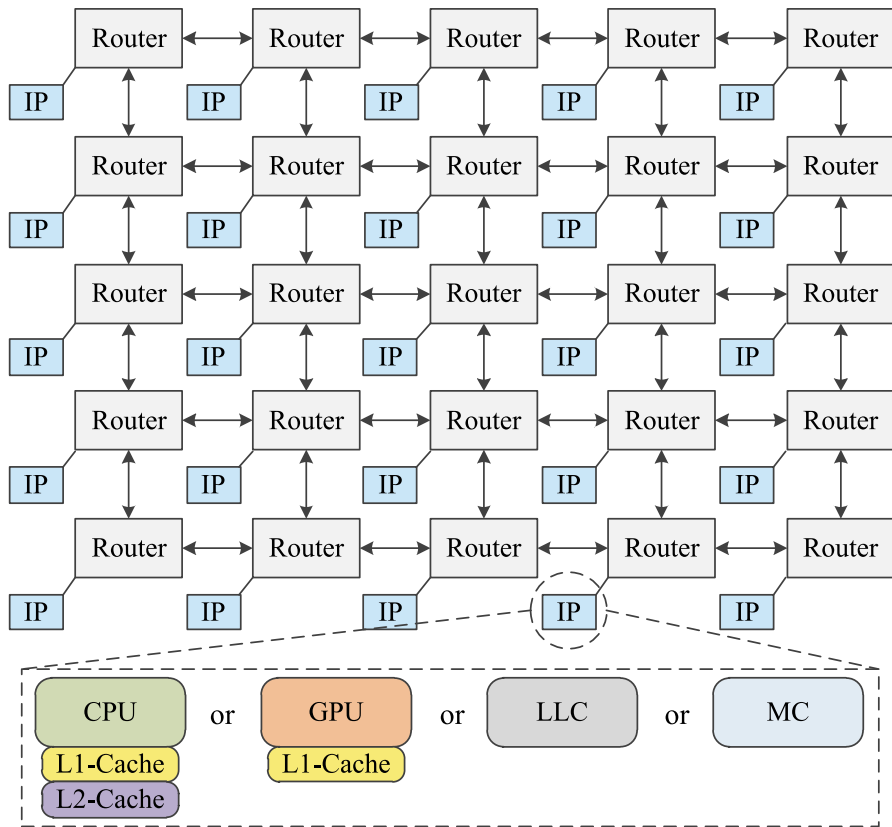


Fig. 1 Heterogeneous NoC architecture

The modularity of the NoC allows for flexible placement of IP nodes based on different tasks. In a heterogeneous system, LLC and MC are shared resources that perform numerous memory access tasks from CPU and GPU and form a critical part of the NoC. Reasonable placement of LLC and MC can reduce memory access path delay and prepare for path division. Based on the location of LLC/MC, we categorize the structure of heterogeneous NoC into three models: center placement (LLC/MC CENTER), side placement (LLC/MC SIDE), and four corner placement (LLC/MC CORNER), as shown in Fig. 2. The LLC/MC SIDE model places LLC/MC only near either CPU or GPU, resulting in significant conflicts at the LLC convergence. Although the LLC/MC CORNER model can disperse traffic conflicts well, it also increases the number of routing hops, which is not conducive to optimization. The LLC/MC CENTER model takes into account the memory access tasks of CPU and GPU, with MC surrounded by LLC to reduce the number of hops. The number of hops in the route for CPU and GPU to access LLC is also reduced to a minimum. However, this model's central location concentrates all traffic, which may lead to congestion. Considering traffic characteristics, LLC should be placed to CPU and

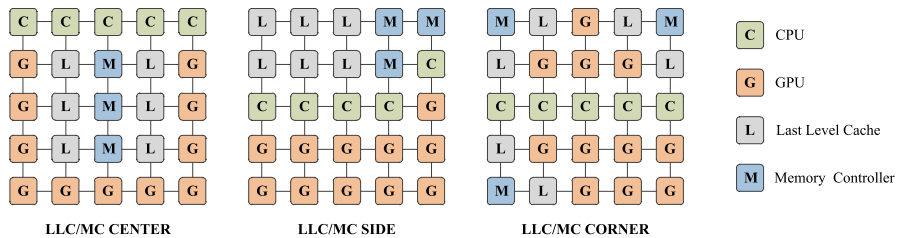


Fig. 2 Heterogeneous NoC with different topologies

GPU as close as possible. Therefore, LLC/MC CENTER is chosen as the baseline model in this paper. In the experiments, we demonstrate the performance of LLC/MC CENTER in comparison with SIDE and CORNER cases.

3.2 Routing algorithm

During on-chip network communication, achieving low latency and high bandwidth is crucial for the entire system. The routing algorithm determines the path that data packets take from the source to the destination in different network topologies, aiming to avoid hotspots and reduce the probability of packet collisions, thus improving overall system bandwidth and reducing latency. By designing routing algorithms reasonably, network latency can be reduced, and system throughput can be increased. However, critical paths and congestion may still occur in routing, requiring the consideration of minimizing path hops to enhance overall system communication capacity. For heterogeneous on-chip networks, unlike traditional homogeneous networks, the impact of core heterogeneity on the network is not negligible. Traditional routing algorithms do not address individual routing algorithm designs for different cores. The number of GPU communication tasks is significantly higher than that of CPU communication tasks, necessitating routing algorithm designs that account for the placement of nodes in different core locations and core characteristics. Moreover, deadlock and load balancing issues should be factored in during the designing of routing algorithms.

3.3 CPU and GPU resource contention

To assess the degree of resource contention between the CPU and GPU in the LLC/MC model, we followed the classification method proposed by Lee et al. [20] and Fang et al. [25] to categorize the benchmark sets. Specifically, we classified the CPU and GPU benchmarks into high-traffic and low-traffic groups, as illustrated in Tables 3 and 4, respectively. For the CPU benchmarks in Table 3, we divided benchmarks whose Packet per Kilo Cycle's (PKC) are greater than 20 into a high-traffic group, and those less than 20 into a low-traffic group. As for the GPU benchmarks in Table 4, we divided benchmarks whose PKC's exceed 100 into a high-traffic group, and those below 100 into a low-traffic group [20]. To evaluate the performance impact on the CPU when running CPU benchmarks only versus running together

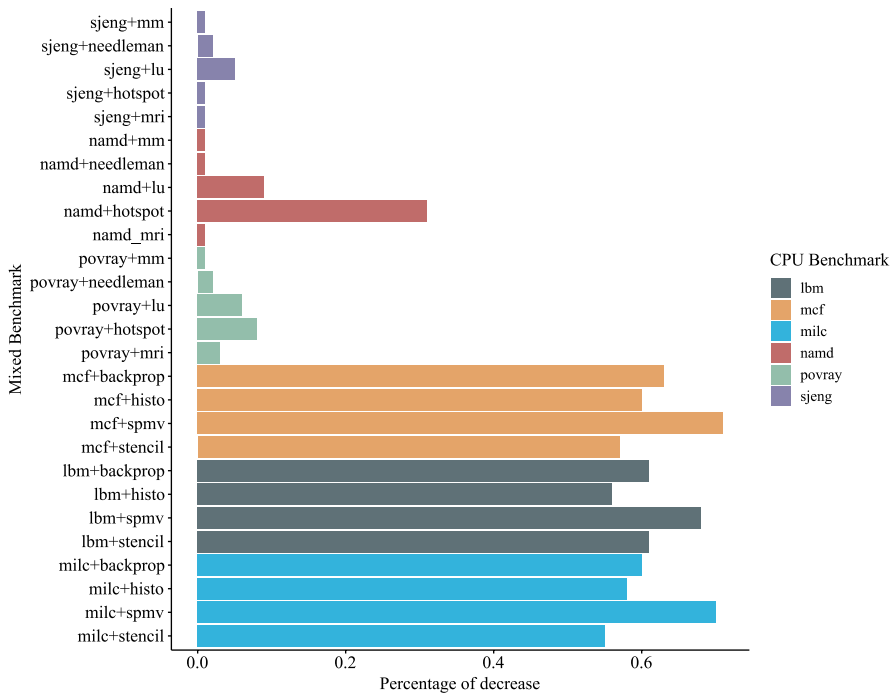


Fig. 3 CPU performance decrease when running CPU and GPU at the same time

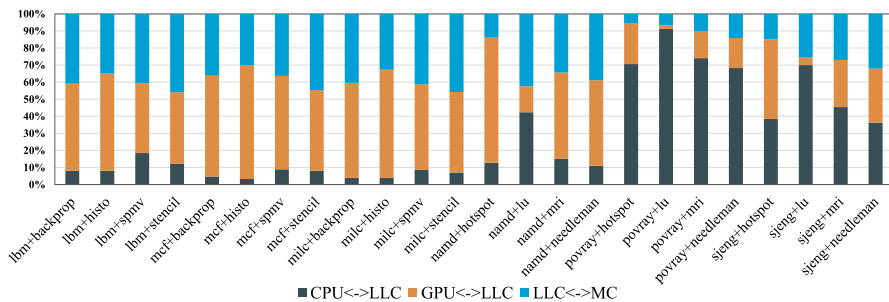


Fig. 4 Proportions of network routing tasks

with GPU benchmarks in the baseline model (LLC/MC CENTER), we compared the results, as depicted in Fig. 3. The figure clearly shows that CPU performance suffers significant degradation under high-traffic conditions, with an average decrease of up to 60% in the high-traffic group, as opposed to only 4% in the low-traffic group.

We also measured the proportions of network routing tasks for 24 groups of mixed loads, as shown in Fig. 4. During periods when network load is relatively low, it is evident that communication frequency between the CPU and LLC accounts for the majority of the load, meaning that CPU tasks are less likely to be preempted by

GPU tasks for resources. For example, in the three groups of mixed loads in the Pov-ray test set, tasks between the CPU and LLC can account for 70% of the total workload. When the network is under high load, the GPU and LLC communicate more frequently. In this case, CPU tasks are often preempted by GPU tasks for resources. On the other hand, the communication frequency between LLC and MC remains relatively constant, but it also accounts for nearly 30% of the workload. Therefore, our optimization of the heterogeneous network communication routing algorithm is twofold: first, to reduce the contention of the GPU for shared resources and improve the accessibility rate of CPU tasks; second, to reduce the path access of hotspots for LLC and MC communication.

4 Heterogeneous NoC routing algorithm

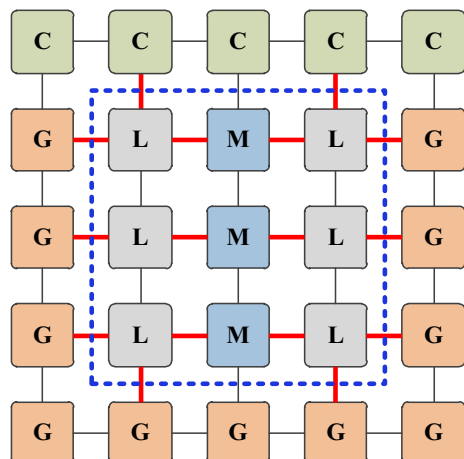
In this section, we analyzed the path flow characteristics of the LLC/MC-CENTER model for heterogeneous NoC and designed dynamic and static routing algorithms on the LLC/MC CENTER model.

4.1 Task-based routing algorithm(TB)

Regardless of the routing algorithm used, the LLC/MC CENTER model will result in a highly congested path due to the direct access of the CPU and GPU to the LLC. Furthermore, frequent communication between the LLC and MC will create hot spots in the center of the network. Figure 5 illustrates this issue, where the congested path is represented by the red line and the hotspot areas are circled by dotted lines.

In Fig. 5, we can clearly observe the flow characteristic of the LLC/MC CENTER model. Since there is no direct communication between individual LLCs and MCs, the traffic in the vertical path within the hotspot area will be lower than that in the horizontal path. As for the outer area, the traffic on the ring path will be lower than the traffic in the middle hotspot area. Based on the

Fig. 5 Hotspot areas and high-traffic paths for LLC/MC CENTER model



structure's characteristics, we can allocate traffic in a reasonable manner to reduce network congestion. We propose a four-point path selection rule with decreasing priority from top to bottom, described as follows:

1. Execute tasks with the least number of hops possible.
2. Assign as few high-traffic paths as possible.
3. Exit the hotspot area as quickly as possible.
4. Distribute workload evenly across the network.

Regarding routing algorithms, the XY routing algorithm is the most commonly used in NoC. It is a deadlock-free and highly reliable algorithm under low workload. The XY routing algorithms prohibit certain turns to avoid loop dependencies. Specifically, only data packets moving east (or west) are allowed to turn to those moving north (or south), thereby avoiding loops. However, under high workload, the problem of path delay becomes more serious.

Algorithm 1 TB Routing Algorithm

Require: *src* is source node, *dst* is destination node, *src.x* is source node horizontal coordinate, *src.y* is source node vertical coordinate

- 1: **if** CPU request tasks **OR** GPU request tasks **OR** GPU reply tasks **then**
- 2: **if** *src.x* > *dst.x* **then** choose straight route to the **LEFT**
- 3: **else if** *src.x* < *dst.x* **then** choose straight route to the **RIGHT**
- 4: **else if** *src.y* > *dst.y* **then** choose straight route to the **UP**
- 5: **else if** *src.y* < *dst.y* **then** choose straight route to the **DOWN**
- 6: **end if**
- 7: **end if**
- 8: **if** CPU reply tasks **OR** tasks between LLC and MC **then**
- 9: **if** *src.y* > *dst.y* **then** choose straight route to the **UP**
- 10: **else if** *src.y* < *dst.y* **then** choose straight route to the **DOWN**
- 11: **else if** *src.x* > *dst.x* **then** choose straight route to the **LEFT**
- 12: **else if** *src.x* < *dst.x* **then** choose straight route to the **RIGHT**
- 13: **end if**
- 14: **end if**

Based on the aforementioned principles, when performing a cache access request task (CPU->LLC/GPU->LLC), the dimension-order XY routing algorithm is better suited to allow the CPU and GPU to access the hotspot area. When performing CPU reply (LLC->CPU), as depicted in Fig. 6, the vertical path in the hotspot area has fewer high-traffic paths, and there is also little traffic on the lateral path in the CPU area. For CPU reply, the dimension-order YX routing algorithm is chosen to avoid potential congested paths. When performing GPU reply (LLC->GPU), the XY routing algorithm is selected since most GPUs can exit the hotspot area quickly, as there is less traffic in the vertical and circular paths. To balance the network load for routing tasks in hotspot areas, we adhere to rule 4)

4.2 TB-TBP routing algorithm

To further enhance the performance of the heterogeneous architecture, we propose the Task-Based-Partition Algorithm(TBP), which entails assigning each virtual channel to execute a particular task. As illustrated in Fig. 8, VC1 is dedicated to executing all tasks of the XY routing algorithm, while VC2 is responsible for executing all tasks of the YX routing algorithm. Both routing algorithms utilized by the virtual channels are deadlock-free. While the task-based routing algorithm distributes XY and YX routing algorithms in the same dimension, the advantage of the TBP routing algorithm is that it reduces conflicts between CPU and GPU by separating different tasks in different dimensions. However, this comes at the cost of utilization of virtual channels, leading to a reduction in network throughput to some extent, but ultimately resulting in improved performance.

Although the TB routing algorithm can effectively avoid path conflicts and resource contention, the escape virtual channel is not well utilized in high-load network situations and is only used as a backup channel to avoid deadlocks. If the escape virtual channel is used reasonably, congestion problems can be further reduced. The TBP routing algorithm makes good use of virtual channel resources but wastes on-chip resources at lower network loads. Therefore, we propose the TB-TBP adaptive routing algorithm for improvement. The TB-TBP routing algorithm combines the advantages of the TB and TBP routing algorithms and dynamically monitors the phase behavior information of application programs in the network through monitoring technology. In high-load network situations, it adopts the TBP algorithm to improve network communication efficiency, while in low-load networks, it adopts the TB algorithm to avoid wasting channel resources and improve network throughput, as shown in Fig. 8.

In order to dynamically monitor the core performance, we propose a dynamic monitoring technique that regards the number of CPU core's retired instructions as a measure of core performance. The variation in the retired instruction count is used as a parameter to dynamically switch between the TB and TBP routing algorithms.

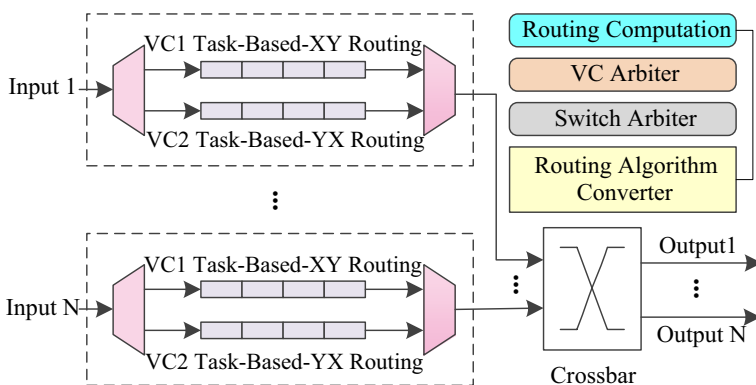


Fig. 8 The use of TBP in the TB-TBP algorithm during dynamic processes

This is consistent with the research methodology employed by previous researchers [20, 26]. We selected the LLC/MC CENTER model mentioned above and used the TB routing algorithm to determine the number of retired instructions for different CPU benchmarks under the same GPU benchmark. Then, we plotted the trend of retired instruction count per hundred cycles, as shown in Fig. 9.

Each CPU application exhibits its own unique phased behavior, and there is no regularity to the number of retired instructions. Retired instructions are the number of instructions processed by the CPU in a unit time. Therefore, the better the performance of the CPU, the higher the number of retired instructions in a unit time, which proves its rapid processing ability. The lower the number of retired instructions per unit time, the slower the processing speed of the CPU during that unit of time, and therefore it can be used as an indicator to dynamically reflect the performance of the CPU. From the blue trend chart, it can be observed that each application may have a

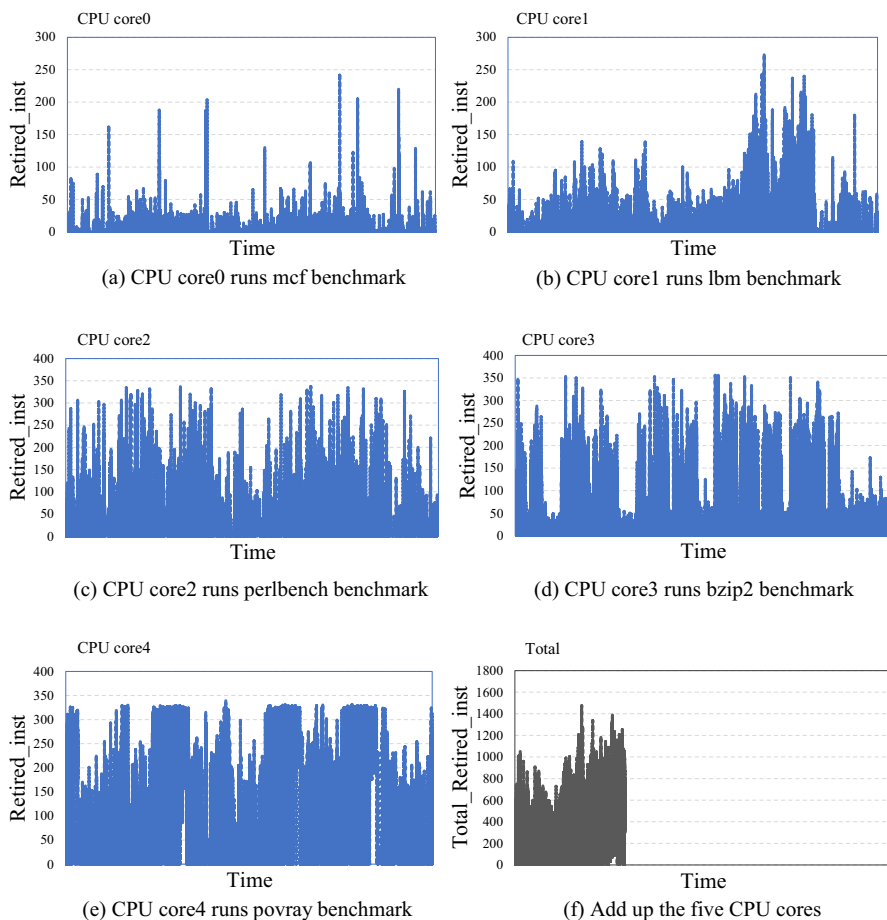


Fig. 9 Retired instructions of different CPU cores and the total

high number of retired instructions at different time periods, or the overall number of retired instructions may be low. For example, the mcf benchmark program run by core0 has many memory access tasks and a high PKC, which can easily cause network congestion issues. Therefore, the overall number of retired instructions is also low, as the number of instructions processed by the CPU per unit time is low. However, when we consider all applications as a whole, as shown in the black trend chart in the lower right corner, the phased behavior of each application becomes blurred and indistinct. Under the overall context, the applications exhibit relatively periodic behavior, which provides a basis for our subsequent optimization. When we bind different applications to different CPU cores, the number of retired instructions can be tracked through dynamic monitoring.

Accurate switching between states can better improve network performance, otherwise, there may be a situation where system overhead increases but performance improvement is not significant. By observing the changes in the number of retired instruction, we found that if the routing algorithm is switched in a short period of time, accurate information about the number of retired instruction cannot be obtained, affecting the judgment of algorithm switching. However, if the routing algorithm is switched after a long time, it cannot efficiently improve the system's performance. Hence, it is crucial to identify the appropriate sampling and main periods that yield maximum benefits, as depicted in Fig. 10.

We set up three types of periods:

The start-up period: During the start-up period, the system state is unstable, and we use the TB algorithm. No sampling work is performed during this period.

The sampling period: The purpose of the sampling period is to obtain information about the current performance of the routing algorithm through sampling, in order to decide whether to switch to a different routing algorithm. When the TB algorithm is used, we switch to the TBP routing algorithm promptly when the CPU's retired instruction count is low, allowing the YX routing algorithm's traffic to enter the escape virtual channel. When utilizing the TBP routing algorithm, it is deemed necessary to transition toward the TB routing algorithm for the purpose of achieving a balanced network load in situations When there is less network congestion. The decision to switch between routing algorithms is determined through the application of formula 1, which calculates the speedup ratio of the two algorithms before and after the switch. Based on the analysis of the information in Fig. 9, the average number of retired instructions

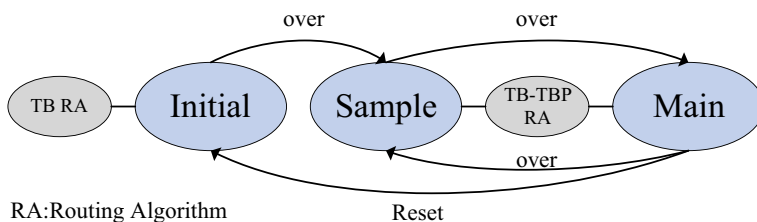


Fig. 10 State transition in heterogeneous NoC

at the peak is approximately 1.5 times that at the valley. Therefore, we separated the two periods by setting a threshold. In practical experiments, when we set the ratio of the two types of retired instructions to 1.5 as the standard to switch routing algorithms during the training period, the network experienced significant pressure relief, indicating that it recognized the period of low retired instructions and adopted the TBP algorithm. Therefore, we set the threshold to 1.5. Specifically, when the speedup ratio surpasses or equals to a threshold level of 1.5, the TB routing algorithm is preferentially selected. In contrast, when the speedup ratio falls below said threshold, the TBP routing algorithm is deemed more suitable for optimizing network performance.

$$\text{Speedup}_{\text{Routing Algorithm}} = \frac{\sum \text{inst_retired}_{\text{TB}}}{\sum \text{inst_retired}_{\text{TBP}}} \quad (1)$$

The main period: it uses the better-performing routing algorithm from the sampling period to maintain stable system performance over an extended period. Once the main period is over, we enter the sampling period to begin sampling again. The duration of each period is specified in Table 1.

5 Experiment setup

In this section, we list the experimental configuration, benchmark sets, and evaluation metrics.

5.1 System setup

We use MacSim version 2.3 [27] as the simulator for the experiments. To better measure the power consumption of routing algorithms, we use the ORION [28] model in MacSim. MacSim is a heterogeneous architecture simulator that is trace-driven and cycle-level. MacSim supports X86 and NVIDIA PTX instruction set architectures. The architectures simulated in this experiment are Intel's Sandy Bridge and NVIDIA's Fermi architecture. Table 2 shows our emulator configuration and NoC configuration. For multi-application emulation, an early terminated application sometimes must be re-executed to simulate resource contention (cache, on-chip interconnect, and memory controller) until all applications are complete. The

Table 1 Period length setting

Period name	Period length
Start-up period	700K cycles
Sampling period	200K cycles
Main period	TBP: 2 M cycles; TB: 3 M cycles

Table 2 Heterogeneous CPU-GPU architecture configuration

CPU core	5 cores 3 GHz 4-wide out-of-order(ooo) gshare branch predictor 8-way,64 KB L1 D/I cache, 3-cycle 8-way,256 KB L2 cache, 8-cycle
GPU Core	11 cores 1.5 GHz in-order(io) 2-way 16 SIMD width 8-way, 32 KB L1 D (2 cycle), 8-way 4 KB L1 I (1 cycle) 16 KB software managed cache
LLC	6 tiles (each tile:32-way 1 MB) 64 B line, LRU
Memory	DDR3-1333, 3 MCs (each 8 banks, 2 channels) 41.6 GB/s bandwidth, 2 KB row buffer, FR-FCFS scheduler
NoC	1 GHz, 5*5 2D mesh topology 4 stage(IB, RC, VCA, SA/ST),credit-based VC 2 per port, each VC can hold 5 flits Ports 5 per router 128 bits (16 B) with 1-cycle latency

Table 3 CPU benchmark

PKC <(Low)			PKC >(High)		
Benchmark	Suite	PKC	Benchmark	Suite	PKC
Povray	fp	2	mcf	int	22
Namd	fp	3	bwaves	fp	40
Sjeng	int	3	milc	fp	40
Gamess	fp	3	leslie3d	fp	45
Gobmk	int	6	lbm	fp	68
Bzip2	int	11			
Perlbench	int	15			

Table 4 GPU benchmark

PKC<100(Low)			PKC>100(High)		
Benchmark	Suite	PKC	Benchmark	Suite	PKC
Hotspot	Rodinia	2	Stencil	Parboil	107
Mri	Parboil	3	Backprop	Rodinia	111
luDecomposition (lu)	Rodinia	4	Histo	Parboil	199
NeedlemanWunsch	Rodinia	23	Spmv	Parboil	467
MonteCarlo	SDK	38			
Mm	Parboil	44			
Srad	Rodinia	68			

GPU application will continue to run repeatedly until the CPU application finishes running, in order to simulate the resource contention behavior on the network. This method is consistent with the work of Lee et al [19].

5.2 Benchmark

We utilized the spec 2006 CPU benchmark [29] and several suites of CUDA GPGPU benchmark, including Nvidia CUDA SDK, Rodinia [30], and Parboil [31] in our experiments. Each CPU runs one CPU application, while all GPUs run one GPGPU application. To generate spec 2006 trace, we used pinpoint [32], and GPU Ocelot [33] was used to generate GPGPU benchmark trace. PKC was used as a statistical indicator to evaluate the communication status of the network and determine the high and low traffic groups of applications. The grouping results are presented in Tables 3 and 4.

5.3 Metrics

We have selected IPC (Instructions Per Cycle) as the performance indicator for the CPU. Additionally, we calculate the total network delay while running the application to evaluate the overall performance of the network. The formula for IPC is given by equation 2, where *cycles* represents the number of cycles used by the CPU to execute the application program, and *instruction_i* represents the number of instructions executed by the CPU. Equation 3 gives the formula for calculating the overall IPC of the network, where *n* is the total number of CPU cores.

$$\text{IPC}_i = \frac{\text{instruction}_i}{\text{cycles}} \quad (2)$$

The average IPC of the system is then:

$$\overline{\text{IPC}} = \frac{\sum_{i=0}^{n-1} \text{IPC}_i}{n} \quad (3)$$

The power consumption of NoC refers to the total energy used by NoC. The energy of NoC mainly consists of link energy and router energy. The link energy is divided into dynamic link energy and static link energy. The dynamic link energy is proportional to the distance between the wire, approximately the product of the energy per millimeter of wire ($E_{\text{wire_dynamic_mm}}$), the channel width *w*, and the length of the wire between the two points of the microchip per hop (*Dis_f*). Static link energy is proportional to the length of the NoC's wires (*Link_Length_{NoC}*). The router energy is the sum of buffer energy, crossbar energy, and arbiter energy.

$$E_{\text{link_dynamic}} = E_{\text{wire_dynamic_mm}} \times w \times \sum_{f=0}^{F-1} \text{Dis}_f \quad (4)$$

$$E_{\text{link_static}} = E_{\text{wire_static_mm}} \times \text{Link_Length}_{\text{NoC}} \times \text{cycles} \quad (5)$$

$$E_{\text{link}} = E_{\text{link_dynamic}} + E_{\text{link_static}} \quad (6)$$

$$E_{\text{router}} = E_{\text{buffer}} + E_{\text{crossbar}} + E_{\text{arbiter}} \quad (7)$$

$$E_{\text{NoC}} = E_{\text{link}} + E_{\text{router}} \quad (8)$$

6 Results and analysis

In this section, we assess the path latency across different topologies and investigate the properties of networks with high and low traffic. Additionally, we evaluate the proposed task-based routing algorithm and TB-TBP routing algorithm, and analyze the experimental results.

6.1 LLC/MC CENTER model

Figures 11 and 12 provide latency comparisons among the LLC/MC SIDE, LLC/MC CORNER, and LLC/MC CENTER models when running benchmarks for both high-traffic and low-traffic groups. The results indicate that, for both groups, the LLC/MC CENTER model has the lowest latency among the three placements and is significantly better than the LLC/MC SIDE and LLC/MC CORNER models. This finding aligns with our prior analysis. Thus, we selected the LLC/MC CENTER model as our target model for subsequent experiments.

6.2 Routing algorithm analysis

The comparison algorithms we used include the classic odd-even routing algorithm [34], X-Y dimension order routing algorithm [35], and SD (static and dynamic) routing algorithm [36]. The SD routing algorithm is a hybrid algorithm that adopts

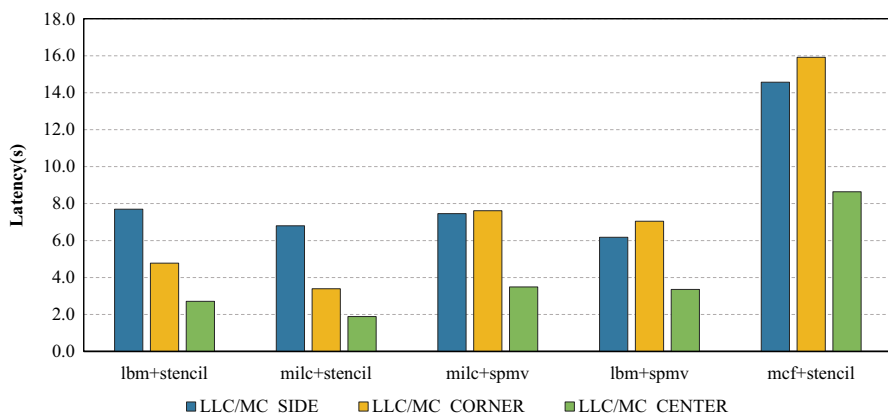


Fig. 11 Latency of high-traffic networks under different topologies

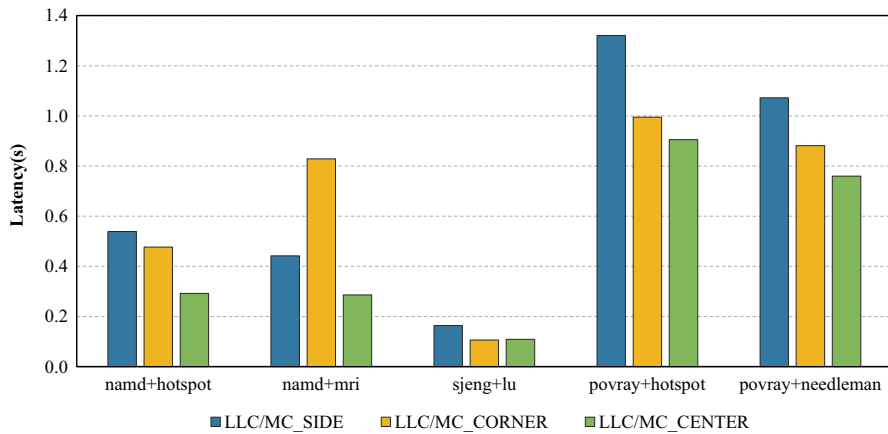


Fig. 12 Latency of low-traffic networks under different topologies

shortest path adaptive routing in congested central node regions and XY routing algorithm in non-central nodes. Figures 13 and 14 demonstrate the latency and IPC comparisons between these three typical routing algorithms and the proposed task-based routing algorithm in a low-traffic network. When running 10 groups of mixed applications, the task-based routing algorithm exhibits an average latency reduction of 1.53% compared to the traditional XY routing algorithm. Additionally, the task-based algorithm shows an average IPC increase of 1.21% compared to the traditional routing algorithm. In a low-traffic network, where there are fewer resource contention conflicts between cores, the algorithm's improvement is not significant, which aligns with our expectations.

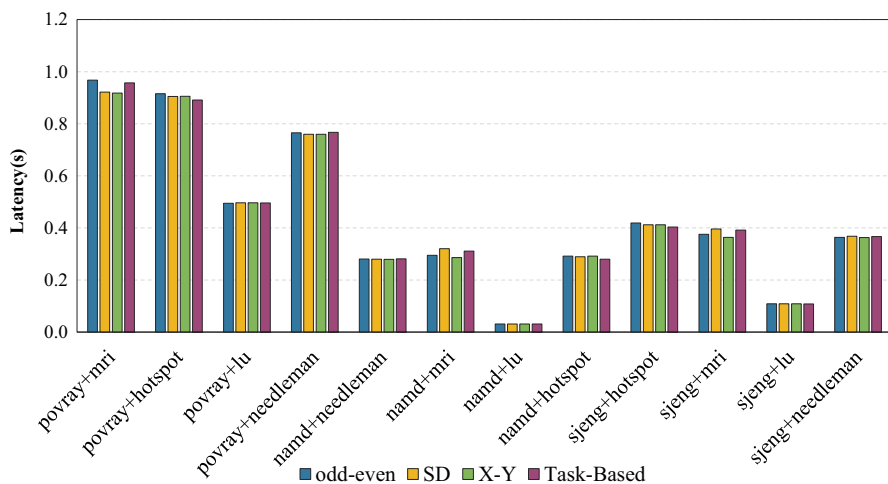


Fig. 13 Latency of different algorithms under low-traffic networks

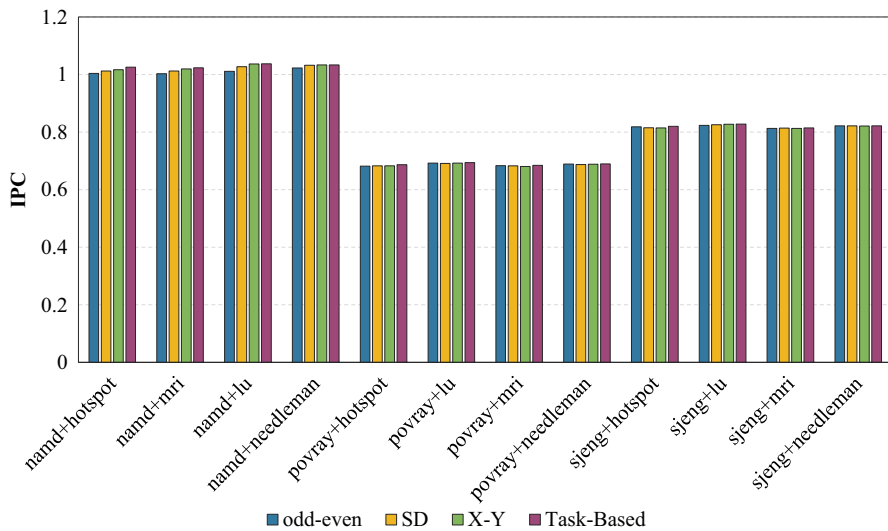


Fig. 14 IPC of different algorithms under low-traffic networks

Under high-traffic conditions, the task-based routing algorithm is evaluated. Among the compared algorithms, the odd-even routing algorithm does not consider the routing characteristics of the network topology, resulting in lower performance compared to other routing algorithms. The SD routing algorithm takes into account the network's tendency to form hotspots in central regions but does not effectively integrate routing selection for different CPU and GPU tasks. In comparison, the task-based routing algorithm exhibits an average latency reduction of 9% compared to the classic XY routing algorithm. For memory-intensive benchmarks such as mcf, the proposed Task-Based algorithm achieves better latency reduction. On average, the task-based routing algorithm reduces the latency of the benchmark by 12%, indicating that our proposed Task-Based routing algorithm is more suitable for CPU-memory-intensive application scenarios.

In terms of CPU performance, under ten groups of mixed loads, the task-based routing algorithm exhibits an average IPC increase of 13.6% compared to the traditional routing algorithm, as depicted in Fig. 15. Similarly, the performance improvement is particularly notable for memory-intensive benchmarks. These findings demonstrate that adopting a reasonable path planning method in high-traffic networks can significantly reduce resource contention and waiting time between the CPU and the GPU. The task-based routing algorithm leverages the network's characteristic of having less traffic on the vertical path to enable faster communication between the CPU and LLC. This mitigates the impact of a large number of GPU tasks and enhances CPU performance. Simultaneously, while our focus was on improving CPU performance, the overall network latency was also reduced accordingly. This indicates that reducing congestion events improves the routing efficiency of both the CPU and GPU on the network simultaneously (Figs. 16, 17 and 18).

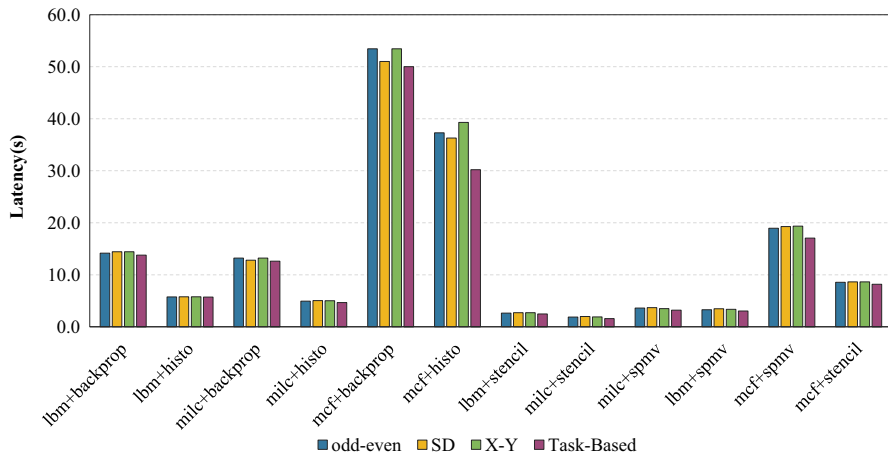


Fig. 15 Latency of different algorithms under high-traffic networks

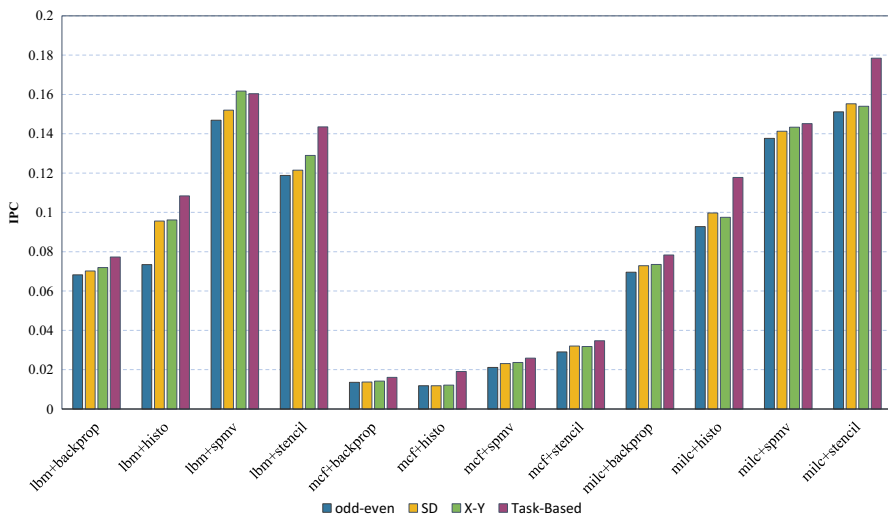


Fig. 16 IPC of different algorithms under high-traffic networks

Furthermore, we introduce the TB-TBP dynamic routing algorithm. Since TB-TBP focuses more on the overall characteristics of the network when running different benchmarks, it better approximates the real system state. Therefore, the testing approach for benchmarks slightly differs from the Task-Based algorithm. We bind different benchmarks to various CPU cores and mix them with different GPU test programs. These CPU benchmarks share the common characteristics described in the previous context. “Mixed-high” refers to different CPU

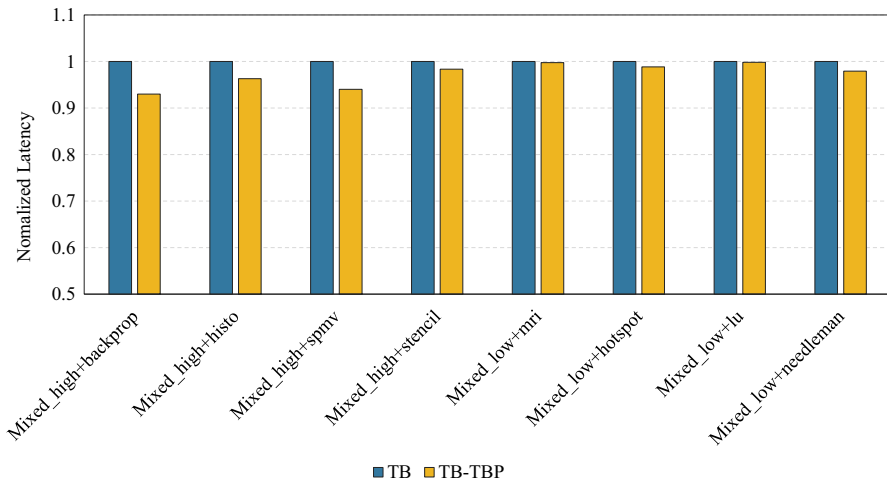


Fig. 17 Latency between TB algorithms and TB-TBP algorithms on network

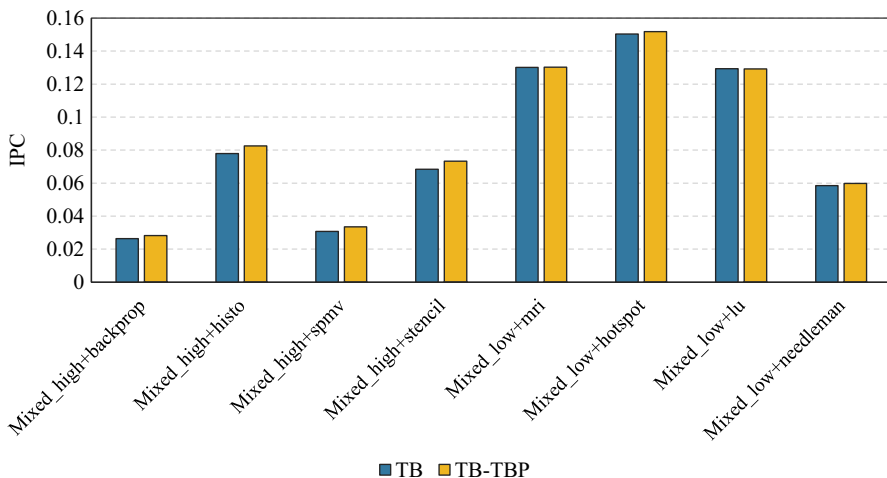


Fig. 18 IPC between TB algorithms and TB-TBP algorithms on network

benchmarks in the high-traffic group, while “Mixed-low” refers to CPU different benchmarks in the low-traffic group. We compare the TB-TBP dynamic routing algorithm with the proposed TB algorithm. In the experiment, under a total of eight mixed loads with both high-traffic and low-traffic groups running, the IPC of the TB-TBP algorithm increased by 4.08% and the latency decreased by 2.74%. The TB-TBP dynamic routing algorithm effectively utilizes virtual channel resources during network peak periods, resulting in reduced network congestion.

6.3 Energy consumption

In this part, we analyze the power consumption of the proposed TB routing algorithm and TB-TBP routing algorithm on the LLC/MC CENTER topology model. We compared the network power consumption of the XY routing algorithm with that of the TB routing algorithm. Compared to the XY routing algorithm, the TB routing algorithm increases power consumption by an average of 0.8%, with the largest group of benchmarks increasing by 1.4%. Compared to the XY routing algorithm, the power consumption will slightly increase because an additional escape virtual channel is used to avoid the deadlock problem. Because the problem of deadlock does not often occur on the network, and the use of escape virtual channels is not long-term, the power consumption of TB routing does not increase significantly compared to XY routing.

On the other hand, the TB-TBP routing algorithm increases power consumption by an average of 3.8% compared to the XY routing algorithm. Compared to the TB routing algorithm and the XY routing algorithm, the TB-TBP routing algorithm occupies virtual channel resources for a longer time, which is also reflected in the increase of power consumption data. Under high-traffic benchmarks, the TB-TBP algorithm consumes more static power, resulting in increased power consumption. However, compared to the average increase in IPC of the XY routing algorithm, it is still within an acceptable range. If the timing of switching between TB and TBP routing algorithms can be better mastered, further power loss can be reduced. Additionally, the proposed two algorithms reduce delay and eliminate heat generation to some extent, which can also reduce power loss.

7 Conclusion

In this study, we addressed the placement problem of core components, namely shared last-level cache (LLC) and memory controller (MC), in heterogeneous on-chip networks, as their proper placement can significantly improve network performance. Placing LLC/MC in the middle of the network allows for the separation of CPU and GPU traffic accessing LLC. However, the middle placement of LLC/MC can result in the formation of hotspots, which leads to network congestion. To address this congestion issue, we designed the task-based routing algorithm based on the tasks in the heterogeneous on-chip network and developed the TB-TBP dynamic adaptive routing algorithm by analyzing network communication characteristics. Finally, extensive simulations are conducted to verify the performance of our proposed algorithm. By running mixed benchmarks, the task-based routing algorithm demonstrated a 9% reduction in overall network latency compared to traditional routing algorithms, while improving CPU performance by 13.6%. The TB-TBP routing algorithm achieved a 2.74% reduction in overall network latency and a 4.08% improvement in CPU performance compared to the task-based routing algorithm. Compared with traditional routing algorithms, the power consumption of the two algorithms increased by an average of 0.8% and 3.8%, respectively. The TB-TBP algorithm proposed in this statement is applicable to all central structures of LLC/MC CENTER, not limited to the 5*5 topology scale of the paper,

and can effectively solve the problem of central hotspot areas. After the expansion of the scale, the placement of individual nodes may be slightly different from the model we propose. However, with the ideas proposed in this paper, it is possible to divide tasks and virtual channels well and choose appropriate task-based routing algorithms. In future work, we aim to further enhance the accuracy of congestion monitoring in the TB-TBP algorithm and determine optimal algorithm cycle lengths. Additionally, we plan to scale up the number of CPU and GPU cores in the network to evaluate the performance of the algorithms in larger network architecture.

Appendix A Acronym Table

The acronyms and their corresponding complete spelling are shown in the table in the appendix.

Acronym	Complete Spelling
NoC	Network-on-chip
LLC	Last-level cache
MC	Memory controller
VC	Virtual channel
PKC	Packet per kilo cycle
IPC	Instruction per cycle
TB	Task-based
TBP	Task-based-partition

Author contributions ZCW and JF designed the TB and TB-TBP routing algorithms for heterogeneous network-on-chip, and YL participated in the experimental testing and analysis of MacSim benchmark. YH participated in the optimization of image design and text in the paper. All authors composed the rest of the manuscript, reviewed the whole manuscript, and approved the final manuscript.

Funding This work is supported by Beijing Natural Science Foundation (4192007) and supported by the National Natural Science Foundation of China (61202076), along with other government sponsors.

Data availability All data generated or analyzed during this study are included in this article. There are no further materials to provide.

Declarations

Conflict of interest The authors declare that they have no competing interests.

Consent for publication The authors readily consent to have this paper published.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line

to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Fang J, Yu L, Liu S, Lu J, Chen T (2015) Kl_ga: an application mapping algorithm for mesh-of-tree (mot) architecture in network-on-chip design. *J Supercomput* 71:4056–4071
2. Rotem E, Naveh A, Ananthakrishnan A, Weissmann E, Rajwan D (2012) Power-management architecture of the intel microarchitecture code-named sandy bridge. *IEEE Micro* 32:20–27
3. Lee KS, Lin H, Feng W-c (2013) Performance characterization of data-intensive kernels on amd fusion architectures. *Comput Sci Res Dev* 28:175–184
4. Matoussi O (2021) Noc performance model for efficient network latency estimation. In: 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp 994–999
5. Ma S, Wang Z, Liu Z, Jerger NDE (2015) Leaving one slot empty: flit bubble flow control for torus cache-coherent NoCs. *IEEE Trans Comput* 64:763–777
6. Marculescu R, Ogras ÜY, Peh L-S, Jerger NDE, Hoskote YV (2009) Outstanding research problems in NoC design: system, microarchitecture, and circuit perspectives. *IEEE Trans Comput Aid Des Integr Circuits Syst* 28:3–21
7. Cong J, Gill M, Hao Y, Reinman GD, Yuan B (2015) On-chip interconnection network for accelerator-rich architectures. In: 2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC), pp 1–6
8. Zheng H, Wang K, Louri A (2020) A versatile and flexible chiplet-based system design for heterogeneous manycore architectures. In: 2020 57th ACM/IEEE Design Automation Conference (DAC), pp 1–6
9. Cheng X, Zhao Y, Zhao H, Xie Y (2018) Packet pump: overcoming network bottleneck in on-chip interconnects for gpgpus*. In: 2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC), pp 1–6
10. Kim HK, Kim J, Seo W, Cho Y-G, Ryu S (2012) Providing cost-effective on-chip network bandwidth in gpgpus. In: 2012 IEEE 30th International Conference on Computer Design (ICCD), pp 407–412
11. Lee YS, Kim YW, Han TH (2023) Mrcn: throughput-oriented multicast routing for customized network-on-chips. *IEEE Trans Parallel Distrib Syst* 34:163–179
12. Khodadadi E, Barekatain B, Yaghoubi E, Mogharrabi-Rad Z (2021) Ft-pdc: an enhanced hybrid congestion-aware fault-tolerant routing technique based on path diversity for 3d NoC. *J Supercomput* 78:523–558
13. Alaei M, Yazdanpanah F (2021) A high reliable multicast routing algorithm for 2d and 3d mesh-based NoCs with fuzzy-based load control. *J Control* 15(1):113–125
14. Salamat R, Khayambashi M, Ebrahimi M, Bagherzadeh N (2018) Lead: an adaptive 3d-NoC routing algorithm with queuing-theory based analytical verification. *IEEE Trans Comput* 67:1153–1166
15. Salamat R (2018) Design and evaluation of high-performance and fault-tolerant routing algorithms for 3d-NoCs
16. Charles S, Mishra P (2020) Lightweight and trust-aware routing in NoC-based SoCs. In: 2020 IEEE computer society annual symposium on VLSI (ISVLSI), pp 160–167
17. Tang M, Lin J, Palesi M (2018) The suboptimal routing algorithm for 2d mesh network. *IEEE Trans Comput* 67:704–716
18. Kao S-C, Yang C-HH, Chen P-Y, Ma X, Krishna T (2019) Reinforcement learning based interconnection routing for adaptive traffic optimization. In: Proceedings of the 13th IEEE/ACM international symposium on networks-on-chip
19. Lee J, Li S, Kim H, Yalamanchili S (2013) Design space exploration of on-chip ring interconnection for a CPU-GPU heterogeneous architecture. *J Parallel Distrib Comput* 73:1525–1538
20. Lee J, Li S, Kim H, Yalamanchili S (2013) Adaptive virtual channel partitioning for network-on-chip in heterogeneous architectures. *ACM Trans Des Autom Electron Syst (TODAES)* 18:1–28

21. Cui Y-W, Prabhakar SM, Zhao H, Mohanty SP, Fang J (2020) A low-cost conflict-free NoC architecture for heterogeneous multicore systems. In: 2020 IEEE computer society annual symposium on VLSI (ISVLSI), pp 300–305
22. Li Y, Louri A (2020) Alpha: a learning-enabled high-performance network-on-chip router design for heterogeneous manycore architectures. *IEEE Trans Sustain Comput* 6:274–288
23. Yin J, Zhou P, Sapatnekar SS, Zhai A (2014) Energy-efficient time-division multiplexed hybrid-switched noc for heterogeneous multicore systems. In: 2014 IEEE 28th international parallel and distributed processing symposium, pp 293–303
24. Zhan J, Kayiran O, Loh GH, Das CR, Xie Y (2016) Oscar: orchestrating STT-RAM cache traffic for heterogeneous CPU-GPU architectures. In: 2016 49th annual IEEE/ACM international symposium on microarchitecture (MICRO), pp 1–13
25. Fang J, Leng Z, Liu S, Yao Z, Sui X (2015) Exploring heterogeneous NoC design space in heterogeneous GPU-CPU architectures. *J Comput Sci Technol* 30:74–83
26. Ma S, Lu H, Huang L, Shen L, Guo Y, Wang Z, Xue W (2018) Adaptive vc partitioning for NoCs in gpgpus. In: 2018 IEEE International Symposium on Circuits and Systems (ISCAS), pp 1–5
27. Kim H, Lee J, Lakshminarayana NB, Sim J, Pho T (2012) Macsim: A CPU-GPU heterogeneous simulation framework user guide. Georgia Institute of Technology, pp 1–57
28. Kahng AB, Lin B, Nath S (2015) Orion3.0: a comprehensive NoC router estimation tool. *IEEE Embed Syst Lett* 7:41–45
29. Henning JL (2006) Spec cpu2006 benchmark descriptions. *SIGARCH Comput Archit News* 34:1–17
30. Che S, Boyer M, Meng J, Tarjan D, Sheaffer JW, Lee S-H, Skadron K (2009) Rodinia: a benchmark suite for heterogeneous computing. In: 2009 IEEE international symposium on workload characterization (IISWC), pp 44–54
31. Stratton JA, Rodrigues CI, Sung I-J, Obeid N, Chang L-W, Anssari N, Liu G, Hwu WW (2012) Parboil: a revised benchmark suite for scientific and commercial throughput computing, pp 1–11
32. Patil H, Cohn RS, Charney MJ, Kapoor R, Sun A, Karunanidhi A (2004) Pinpointing representative portions of large intel; itanium; programs with dynamic instrumentation. In: 37th International Symposium on Microarchitecture (MICRO-37'04), pp 81–92
33. Farooqui N, Kerr A, Diamos GF, Yalamanchili S, Schwan K (2011) A framework for dynamically instrumenting gpu compute applications within gpu ocelot. In: GPGPU-4
34. Chiu G-M (2000) The odd-even turn model for adaptive routing. *IEEE Trans Parallel Distrib Syst* 11:729–738
35. An J, You H, Sun J, Cao J (2021) Fault tolerant xy-yx routing algorithm supporting backtracking strategy for noc. 2021 IEEE Intl conf on parallel & distributed processing with applications, big data & cloud computing, sustainable computing & communications, social computing & networking (ISPA/BDCLOUD/SocialCom/SustainCom), pp 632–635
36. Chang-shan W (2009) Sd: a routing algorithm for mesh based network-on-chip. *Microcomput Inf* 25:93–94